

HCML PROJECT CONCEPTUAL NOTES

These notes explain the project in simple language, with the technical terms you can use in the report or presentation.

1. MAIN PROJECT GOAL

The project is about hidden identity recovery from morphed face images.

A morph image is created from two people:

Person A + Person B $\rightarrow$ morphed image M_AB

In our task, one identity is known:

known identity = A

The goal is to recover the other identity:

hidden identity = B

Simple example:

input morph: Alice + Bob

known identity: Alice

target: recover Bob

Presentation sentence:

Quote: Given a morph image and one known contributing identity, the goal is to

Quote: generate a face image that is closer to the hidden contributing identity.

2. DATASET: MAD22

MAD22 contains:

1. Bona fide images: real/genuine source face images
2. Morphed images: images created by combining two identities

Example morph filename:

001_08-vs-010_08.jpg

This means the morph was made from:

001_08 and 010_08

For each morph, we create two directed recovery trials:

Trial 1: morph + 001_08 -> recover 010_08

Trial 2: morph + 010_08 -> recover 001_08

Technical term:

directed hidden-identity recovery trials

Why two trials?

The task is directional. If A is known, B is hidden. If B is known, A is hidden.

3. METADATA CSV

The metadata CSV turns image filenames into clear machine-readable tasks.

Each row stores:

trial_id
morph_path
known_path
hidden_path
known_id
hidden_id

Technical term:

trial metadata

Presentation sentence:

Quote: I converted the dataset filenames into directed metadata rows, where each

Quote: row specifies the morph image, the known identity, and the hidden target

Quote: identity.

4. SMOKE SUBSET

The full OpenCV subset has:

604 usable morphs
1208 directed trials

Before running everything, we use a small smoke subset:

10 morphs
20 directed trials

Technical term:

smoke test

Why?

It lets us quickly check whether paths, preprocessing, embeddings, model loading, and generation work before spending GPU time on the full dataset.

Smoke-test results are not final research results. They are for debugging.

5. IMAGE MANIFEST

The trial CSV repeats images. The same image can appear in many trials.

An image manifest stores each unique image once.

Technical term:

unique image manifest

Difference:

metadata CSV = what recovery tasks to solve
image manifest = what unique image files need processing

Why?

We align and embed each image only once, then reuse it in all relevant trials.

6. FACE ALIGNMENT

Before extracting identity embeddings, face images should be aligned.

Alignment means:

- detect/crop face
- place face consistently
- resize image

For ElasticFaceArc embeddings, we use:

- 112 x 112 face crops

Technical terms:

- face alignment
- ArcFace-style alignment
- face preprocessing

Important distinction:

- 112x112 aligned images -> used for face-recognition embeddings
- 128x128 images -> used by the diffusion autoencoder

7. ELASTICFACEARC

ElasticFaceArc is the face-recognition model we use for identity contexts.

Simple function:

- face image -> ElasticFaceArc -> 512-dimensional identity embedding

Technical terms:

- face-recognition embedding extractor
- identity encoder
- feature extractor

We use ElasticFaceArc to compute:

- positive context = embedding(morph image)
- negative context = embedding(known identity image)
- hidden context = embedding(hidden identity image, for evaluation)

Important correction from the supervisor:

Final identity contexts must use ElasticFaceArc.

FRmodel_FarNeg_CASIA and FRmodel_FarNegAdaptive_CASIA are not used for final conditioning.

ElasticFaceCos is optional for final cosine-similarity evaluation.

8. IDENTITY EMBEDDINGS

An identity embedding is a vector of numbers representing a face identity.

In our project:

embedding size = 512 numbers

If two images are of the same person, their embeddings should be similar.

Technical term:

512-dimensional identity embedding

Similarity is usually measured with:

cosine similarity

9. AUTOENCODER

The autoencoder works with images, not identity embeddings.

It has two parts:

encoder: image -> latent

decoder: latent -> image

Technical terms:

latent autoencoder

VQGAN autoencoder

first-stage encoder/decoder

In our project:

morph image -> autoencoder encoder -> morph latent

generated latent -> autoencoder decoder -> generated face image

Important:

The autoencoder does not create identity embeddings. ElasticFaceArc creates identity embeddings.

10. LATENT SPACE

Latent space is a compressed representation space.

Instead of generating directly in pixel space, the diffusion model works in latent space.

Simple idea:

image space = visible image pixels

latent space = compressed hidden representation

Why use latent space?

It is more efficient and is the setup used by the pretrained diffusion model.

11. FORWARD DIFFUSION / MARKOVIAN NOISING

Before sampling, the morph image is encoded into latent space and noise is added.

Flow:

morph image -> morph latent -> noisy morph latent

Technical terms:

forward diffusion process

Markovian noising chain

$q(x_t | x_0)$

Supervisor instruction:

Use a total of 1000 noising steps to obtain the starting noise.

Use DDIM with 200 steps during sampling.

12. DM_CASIA DIFFUSION MODEL

DM_CASIA is the pretrained identity-conditioned diffusion model.

Simple function:

noisy latent + identity condition -> denoised/generated latent

Technical terms:

conditional denoising diffusion model

latent diffusion model

identity-conditioned diffusion model

We do not train DM_CASIA. We load pretrained weights and use it for sampling.

13. NEGFACEDIFF

NegFaceDiff is a sampling/guidance method, not a newly trained model in this project.

It uses:

positive context = morph embedding

negative context = known identity embedding

weight = 0.5

adapt = false

Simple idea:

move toward the morph identity

move away from the known identity

hopefully reveal the hidden identity

Technical term:

negative identity guidance

fixed negative guidance

14. ADAPTDIFF

AdaptDiff uses the same general idea as NegFaceDiff, but the guidance strength changes over denoising time.

It uses:

positive context = morph embedding
negative context = known identity embedding
weight = 1.0
adapt = true

Technical terms:

adaptive negative guidance
adaptive guidance schedule

15. FULL CORRECT PIPELINE

The full pipeline has two parallel branches.

Identity branch:

morph image -> ElasticFaceArc -> positive identity embedding
known image -> ElasticFaceArc -> negative identity embedding
hidden image -> ElasticFaceArc -> hidden identity embedding for evaluation

Image/latent branch:

morph image
-> autoencoder encoder
-> morph latent
-> forward diffusion/noising
-> noisy morph latent

Sampling branch:

noisy morph latent
+ positive identity embedding
+ negative identity embedding
-> DM_CASIA with NegFaceDiff or AdaptDiff guidance
-> generated latent

Decoding branch:

generated latent -> autoencoder decoder -> generated face image

Evaluation branch:

generated face image -> face-recognition model -> generated embedding
compare generated embedding with hidden and known embeddings

16. WHAT WE ARE TRAINING

In this project, we are not training the main models.

We use pretrained:

ElasticFaceArc
latent autoencoder
DM_CASIA diffusion model

NegFaceDiff and AdaptDiff are sampling strategies applied to the pretrained diffusion model.

Technical term:

inference / sampling with pretrained models

Not training means we are not doing:

loss.backward()
optimizer.step()

Presentation sentence:

Quote: I did not train a new model from scratch. I used pretrained components and
Quote: evaluated negative identity guidance strategies for hidden identity recovery.

17. CURRENT FACE RECOGNITION AND EVALUATION SETUP

Current identity-context model:

ElasticFaceArc with iresnet100 backbone

Used for:

positive context
negative context
hidden context
smoke sanity checks

Current sanity evaluation:

```
cosine(embedding(morph), embedding(known))  
cosine(embedding(morph), embedding(hidden))
```

This is only a sanity check, not the final diffusion evaluation.

Final generated-image evaluation should compute:

```
embedding(generated image)  
embedding(hidden identity)  
embedding(known identity)
```

Then:

```
sim_to_hidden = cosine(generated, hidden)  
sim_to_known = cosine(generated, known)  
margin = sim_to_hidden - sim_to_known
```

Success condition:

```
sim_to_hidden > sim_to_known
```

Optional:

ElasticFaceCos or another face-recognition model can be used as an additional evaluation model, but ElasticFaceArc is the required model for identity contexts.

18. CURRENT PROJECT STATUS

Completed:

- MAD22 metadata creation
- smoke subset creation
- image manifest creation
- face alignment
- ElasticFaceArc embedding extraction
- paired sampling input preparation
- model asset setup
- paired diffusion sampler
- 20-trial smoke generation on Kaggle GPU

Current observation:

- generated images are blurry
- NegFaceDiff and AdaptDiff outputs look very similar
- generated faces do not clearly match hidden identities yet

Next step:

- debug output quality
- then compute generated-image evaluation metrics